PHP FULL STACK DEVELOPER CASE STUDY

YipOnline E-commerce System

A Laravel 12 e-commerce case study with Smarty templates, MySQL persistence, Cloudinary product images, Paystack checkout, admin management, customer order tracking, and test coverage.

Laravel 12 | PHP 8.2 | Smarty | MySQL | Cloudinary | Paystack

BRIEF

Project Objective

Build a basic e-commerce site using Laravel or PHP, with Smarty integration as a bonus, covering storefront, cart, checkout, authentication, and admin order management.

Customer Experience

Customers can browse products, view details, manage cart, checkout with Paystack, and track orders.

Admin Experience

Admins can manage products, view all orders, inspect order details, and review payment history.

Technical Focus

Clean MVC structure, secure validation, role separation, external service integration, and automated tests.

ARCHITECTURE

Application Structure

Laravel MVC

- Controllers handle storefront, cart, checkout, orders, profile, and admin operations.
- Eloquent models manage users, products, product images, orders, and order items.
- Middleware separates admin and customer-only resources.

Smarty View Layer

- Smarty templates live in `resources/smarty`.
- A custom `SmartyRenderer` service shares auth, CSRF, flash, and cart data.
- Plain CSS powers the responsive interface without a frontend build dependency.

Controllers | Models | Middleware | Migrations | Seeders | Feature Tests

CUSTOMER FLOW

Storefront and Checkout

1. Browse active products
2. View product gallery
3. Add available stock to cart
4. Checkout with address
5. Pay and track order

Cart Safety

Stock is validated when adding, updating, and checking out. Users see friendly messages when an item is out of stock.

Order Tracking

Customers have an Orders menu where they can see order status, payment status, totals, and line items.

ADMIN FLOW

Admin Dashboard

Products

Admins can create, view, edit, search, filter, paginate, and delete products when they have not been ordered.

Orders

Admins can see all system orders, update fulfillment status, and inspect attached products.

Payments

Admins can review Paystack references, payment status, paid revenue, and linked orders.

Admin and customer dashboards are separated with middleware, so admins cannot access customer cart resources and customers cannot access admin resources.

INTEGRATIONS

Cloudinary and Paystack

Cloudinary Images

- Product images are uploaded to Cloudinary.
- Multiple product images are stored in a dedicated `product_images` table.
- Product detail pages show an active image and thumbnail gallery.

Paystack Payments

- Checkout initializes a Paystack transaction.
- Callback verifies transaction reference and amount.
- Orders track payment status, reference, authorization URL, and paid date.

SECURITY

Validation and Access Control

Authentication

Users can register, log in, log out, and update their profile details. Email values are validated and unique.

Authorization

Admin and customer access is separated through route middleware and controller checks.

Form Safety

CSRF tokens, Laravel validation, stock checks, and ownership checks protect common workflows.

DATA MODEL

Core Database Tables

Users

Stores customers and admins with an `is_admin` role flag.

Products

Stores product name, slug, description, price, stock, active status, and main image.

Product Images

Stores multiple Cloudinary image URLs and public IDs per product.

Orders

Stores customer details, shipping address, fulfillment status, total, and payment data.

Order Items

Stores product snapshot, quantity, unit price, and line total for each order.

Sessions

Database-backed sessions support Laravel authentication and session cart storage.

QUALITY

Testing Coverage

19

Automated tests currently pass across unit and feature test suites.

108

Assertions cover checkout, orders, admin CRUD, profiles, access control, payments, and stock checks.

0

External payment/image APIs are mocked in tests to avoid real charges or uploads.

Verification command: `php artisan test`

SETUP

Running the Project

Commands

- `composer install`
- `cp .env.example .env`
- `php artisan key:generate`
- `php artisan migrate --seed`
- `php artisan serve`

Demo Accounts

- Admin: `enerjaizenigeria@gmail.com`
- Customer: `francisitafor1@gmail.com`
- Password: `password123`

SUMMARY

Submission Ready

The implementation satisfies the requested e-commerce brief while adding practical production-style touches: role separation, payment verification, image management, validation, user profiles, stock protection, admin reporting, and automated tests.

Product Listing | Cart | Checkout | Authentication | Admin Dashboard | Smarty Bonus